

London Underground “What-If” Simulator UI/UX and Design Document

1. Product Overview

The London Underground “What-If” Simulator is an interactive, full-screen web application that lets users explore hypothetical scenarios on the London Underground (TfL) network. Users can close stations and lines and see in real time how the network graph changes, which routes are blocked and how alternative routes are calculated. The entire experience lives on a single screen: a full-viewport map with floating, overlaid controls.

The product serves two main audiences: students and commuters curious about route resilience, and researchers or analysts who want to model network disruption scenarios. A secondary use-case is simple route planning against live TfL line-status data.

1.1 Who it serves - personas

Persona A - “The curious commuter” (Maya, 27). Uses the tube daily. Wants to quickly see whether her usual journey would still work if her local station closed, and what the alternative would be. Cares about speed and clarity, uses her phone and is not interested in technical detail.

Persona B - “The planner / enthusiast” (Tom, 41). Interested in how the network behaves under disruption. Likes to close whole lines and explore knock-on effects across multiple journeys. Will use a desktop with the sidebar open and experiment extensively.

Persona C - “The assessor / first-time visitor” (Dr. Lee). Has never seen the app. Needs to understand within seconds what it does and be able to produce a meaningful result - a routed journey, a closure scenario without instruction.

These map onto the two interaction modes: Personas A and C lean on routing (find a journey, see it re-route); Persona B leans on What-If (impose closures, observe effects).

1.2 Core user journeys

- **Route Planning** - select a start station and a destination; the app computes and displays the shortest path with a line-by-line breakdown, travel time, and number of changes.
- **What-If Simulation** - enable What-If Mode, then close individual stations or entire lines by clicking them on the map or toggling them in the sidebar. The routing engine recalculates automatically.
- **Network Monitoring** - observe live TfL train positions, line delays, and station closures on the map without triggering any simulation.
- **Station Search** - type a station name to zoom the map to it and optionally use it as a route endpoint.

1.3 Design goals

- **Make the network legible.** Present the London Underground clearly enough that a user can find stations and lines at a glance.
- **Make “what-if” exploration effortless.** Closing a station or line and seeing the effect on a journey should be immediate and obvious.

- **Keep live and hypothetical clearly separated.** The user should always know whether they are looking at the real network (Live) or their own scenario (What-If).
- **Work on phone and desktop.** The same capabilities should be usable on a small portrait screen and a wide desktop.

1.4 Design goals from the moodboard

The original Figma moodboard established the following directional themes, which the implemented design has since expanded on:

- **Data-dense but readable** - inspired by mission-control dashboards and transit system maps.
- **Dark, immersive aesthetic** - the map is the hero; all UI elements float above it with glass-morphism panels.
- **TfL brand alignment** - use the official TfL line colours throughout to keep the map immediately legible to Londoners.
- **Mode duality** - a clear visual distinction between the calm live-monitoring state and the high-alertness What-If simulation state.

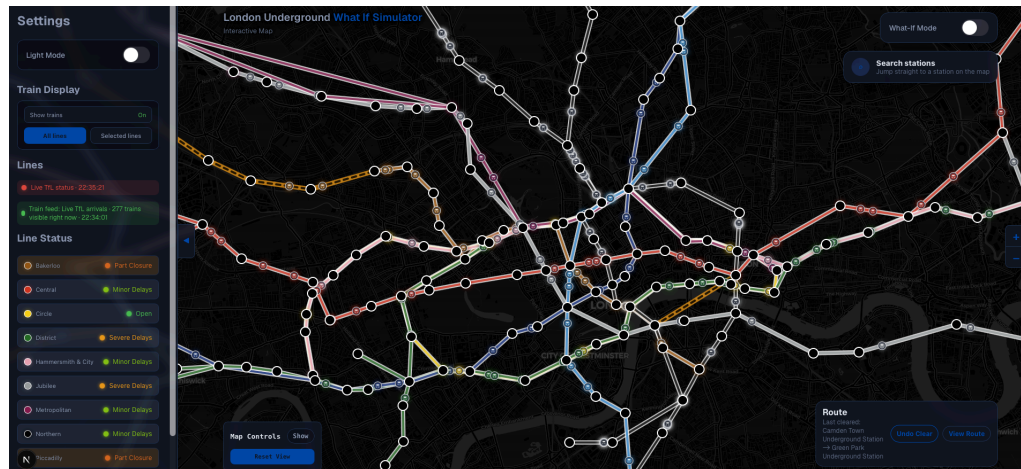
2. Users and Interface-Facing User Stories

The interface is built to satisfy user stories such as:

- *As a user, I can search for a station by name and jump to it on the map.*
- *As a user, I can set a start and a destination and see the fastest journey drawn on the map, with total time and number of changes.*
- *As a user, I can switch into What-If mode and close a station or a whole line.*
- *As a user, when I close part of the network, my planned journey automatically re-routes around it (or tells me no route is possible).*
- *As a user, I can see live train positions moving along the lines.*
- *As a user, I can see which lines currently have delays or closures in Live mode.*
- *As a user on a phone, I can do all of the above through a layout designed for a small portrait screen.*

3. Information Architecture

The application is a single screen: a full-viewport map with overlaid controls. There is no multi-page navigation and no traditional page or route structure - everything happens in context on the map. The functional areas are laid out as follows:



3.1 Primary surfaces

Drawn from components/layout/ and components/mapLayers/:

- **Map title overlay** - names the app and reflects the current mode (colour shifts in What-If mode).
- **What-If toggle**
- **Search box** - station search with keyboard support; selecting a result flies the map to that station and offers set-as-start / set-as-destination / toggle-closure actions.
- **HUD** - shows live zoom level and map centre, with a Reset view button that also clears closures, route and search.
- **Sidebar (desktop) / Control sheet (mobile)** - the main control surface: per-line controls, line status, train visibility filters and live feed status.
- **What-If overlay** - a contextual banner shown when What-If mode is active (desktop).
- **Route info panel** - appears when a journey is computed: ordered stops, the line(s) used, total travel time and number of changes.
- **Selected train panel** - details for a train the user has tapped.
- **Routing error box** - shown when no route exists under the current closures.
- **Mobile interaction-mode bar** - on mobile, switches between “route” and “what-if” interaction.

3.2 Map layers

Station markers (StationLayer), line connections (EdgeLayer), the computed route (RouteLayer), and live trains (TrainLayer / CanvasTrainLayer).

4. Key User Journeys

4.1 Plan a journey

1. User opens the app and sees the full network on a dark map centred on London.
2. User searches for, or taps, a station and sets it as start.
3. User sets a second station as destination.
4. The route is computed (client-side Dijkstra) and drawn on the map; the Route info panel shows stops, lines, total time and changes.

4.2 Run a “what-if” scenario

5. Enables What-If mode (accent colour switches to amber to signal the mode change).
6. User closes a station (tap a marker) or a whole line (toggle in the sidebar). Closed elements are visually marked (red outline / closed-line styling).
7. Any active route re-computes immediately around the closure; if no route is possible the routing error box explains this.
8. User resets via Reset view / Reset closures to return to a clean state.

4.3 Observe live conditions

9. In Live mode, the app overlays real station closures and line status/delays.
The user can toggle live trains on and watch them animate along the lines, and tap one to inspect it.

4.4 Mobile journey

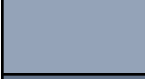
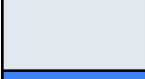
On a phone in portrait, the sidebar is replaced by a bottom control sheet and a mobile interaction-mode bar lets the user switch between routing and what-if interactions, so map taps are interpreted correctly for the current task.

5. Colour System

All colours are defined as CSS custom properties (design tokens) in global.css using the OKLCH colour space, and aliased into the Tailwind theme via tailwind.config.js. This allows consistent theming across both inline JSX styles (which use the hex constants from constants.js) and Tailwind utility classes.

5.1 Core UI palette

The application is dark-first by default

Swatch	Hex	Name and Description	Usage
	#0A0F1A	Background - Deep navy/black	Map container, page background. Prevents flash on tile load.
	#0F172A	Card / Panel - Semi-transparent dark navy	HUD, search box, sidebar, route panel backgrounds (rgba(15,23,42,0.8)).
	#1E3A5F	Border - Muted blue-grey	All panel outlines, dividers, input borders.
	#94A3B8	Text - Slate 400	Primary label text, sidebar content.
	#64748B	Text Muted - Slate 500	Subtitles, timestamps, help text.
	#E2E8F0	Text Light - Slate 200	High-emphasis values - station names, route titles.
	#3B82F6	Accent (Normal) - Blue 500	Active states, selected items, highlight rings - used when What-If Mode is off.

Swatch	Hex	Name and Description	Usage
	#FBBF24	Accent (What-If) - Amber 400	All interactive accents when What-If Mode is enabled, replacing blue.
	#22C55E	Success - Green 500	Live status indicators, open stations/lines, train visibility On state.
	#F59E0B	Warning - Amber 500	Schedule-fallback train feed, partly-closed line status.
	#EF4444	Danger - Red 500	Closed line overlay, Reset Closures button, closed station glow.

5.2 Accent colour duality

One of the most deliberate design decisions is the accent-colour shift. When the user enables What-If Mode, every interactive element (button borders, selection highlights, search results, the sidebar toggle) shifts from blue (#3B82F6) to amber (#FBBF24). This provides an unmistakable, application-wide signal that the user is now in a simulated environment.

The What-If overlay reinforces this with four amber corner brackets and a large monospace “WHAT-IF” label with a pulsing amber dot, rendered over the map viewport. The whole interface shifts colour so the user can never be unsure which mode they are in and the shift is reinforced by text/labelling so it is not conveyed by colour alone.

5.3 TfL official line colours

Line colours are sourced from Transport for London’s official brand guidelines and held in the `LINE_COLOURS` constant in `constants.js`. They are used on map polylines, station dots, train markers and the line-status list in the sidebar. Using the real line colours leverages users’ existing mental model of the tube map.

Swatch	Hex	Line	Notes
	#B36305	Bakerloo	Dark brown. Train marker slightly darker: #AC5A00.
	#E32017	Central	TfL red. Distinct from the danger/error red (#EF4444).
	#FFD300	Circle	Yellow. High-contrast on dark background.
	#00782A	District	Green. Distinct from the success/live-status green.
	#F3A9BB	Hammersmith & City	Light pink.
	#A0A5A9	Jubilee	Silver/grey.
	#9B0056	Metropolitan	Deep magenta/purple.
	#000000	Northern	Black. On dark maps: outlined in white to stay visible.
	#003688	Piccadilly	Dark navy blue.
	#0098D4	Victoria	Sky blue.
	#95CDBA	Waterloo & City	Mint/teal.

6. Typography

The type system uses two Google Fonts loaded via `next/font/google`, which eliminates layout shift and ensures self-hosted delivery. Geist is designed by Vercel and is optimised for on-screen legibility at small sizes which an important property for a data-dense interface.

Variable	Typeface	Role	Used For
<code>--font-sans / font-sans</code>	Geist	Primary	All UI labels, navigation, body text, buttons, station names.
<code>--font-mono / font-mono</code>	Geist Mono	Accent	The WHAT-IF overlay label; HUD coordinate readouts.

6.1 Type scale

No rigid type-scale token system is formalised in code; sizes are set inline in JSX. The recurring scale observed across components is:

CSS px	Usage Context	Example	Notes
10px / 0.625rem	Micro labels	“SEARCH”, “START”, “DESTINATION” uppercase labels	letter-spacing 0.06–0.12em, uppercase
11px	Captions	Status pill text, help text	-
12px	Secondary body	Timestamps, subtitles, stop counts	-
13–14px	Primary body	Search results, route legs, sidebar items	-
15–16px	Headings (small)	Station name in search result card, sidebar h3	-
18px	Panel titles	“Search stations” heading inside search box	fontWeight 800
20px	Map title (desktop)	“London Underground What-If Simulator”	fontWeight 600
40px	What-If overlay	“WHAT-IF” label	monospace, fontWeight 700

6.2 Font-weight conventions

- **fontWeight 400 / 500** - body text, muted labels.
- **fontWeight 700** - button labels, line names, search-result station names.
- **fontWeight 800** - panel titles, route-leg labels, key statistics (Duration, Changes, Stops).
- **What-If overlay** - the WHAT-IF display is set to fontWeight 700 with Geist Mono.

7. Layout System

The layout is fundamentally a full-viewport map with floating overlay panels. There is no traditional page or route structure, the entire application lives on a single screen. State that affects layout (sidebar open/closed, mode, selected train, route panel) is owned by MapCanvas so the panels stay coordinated across breakpoints.

7.1 Viewport and breakpoints

The app uses a custom useViewport hook to derive layout contexts at runtime. There are no CSS media queries used directly; all layout switching is driven by JavaScript state fed into inline styles.

Context	Condition	Impact
isMobilePortrait	isMobile && isPortrait	Activates bottom-sheet control panel, hides desktop sidebar, activates mobile search bar at the top, repositions route panel, shows mobile mode bar.
isLargeDesktop	viewport width > threshold	Sidebar width = 320px (vs. 280px default).

7.2 Z-index stack

All overlay components use fixed/absolute positioning. The z-index layers, from bottom to top, are:

z-index	Layer	Components
0 (Leaflet default)	Map tiles & canvas layers	Tile layer, EdgeLayer, StationLayer, RouteLayer, TrainLayer
999	Ambient overlays	What-If corner brackets, What-If label, map grid/visual chrome
1000	Fixed UI elements	Title overlay, Sidebar, HUD, MapLeafletChrome style injector
1001	Search box	MapSearchBox (desktop and mobile variants)
1100	Sidebar toggle button	SidebarToggleButton
1200	Route panel & error box	RouteInfoPanel, RoutingErrorBox
1300+	Mobile bottom sheet	MobileControlSheet (overlaid via sheet pattern)

7.3 Spacing conventions

Spacing is not tokenised but follows consistent patterns derived from the components:

- **Panel internal padding:** 20px (sidebar), 14px (search box), 12px (route panel, HUD).
- **Gap between panel sections:** 20px (sidebar), 12–16px (route legs, search results).
- **Border radius:** 8px (small rows/pills), 10–12px (control groups), 14–18px (cards/panels), 22px (mobile route panel).
- **Icon dot sizes:** 10px (status indicators), 12–14px (line colour dots in list), 24px (What-If pulsing dot).

8. Component Library

All UI components live under `app/components/layout/` and `app/components/map/`. Map rendering layers are in `app/components/mapLayers/`. UI controls build on Radix UI primitives (dialogs, switches, tooltips, sliders, tabs) styled with Tailwind CSS, with Lucide-react icons and Recharts for charts. Radix primitives were chosen because they ship with accessible behaviour (focus management, ARIA roles, keyboard support) out of the box. Below is a reference for every UI component with its visual characteristics, state variants, and positioning.

8.1 MapTitleOverlay

The application title, always visible in the top-left corner of the map.

Desktop variant: plain text floating directly over the map with a heavy multi-layered text-shadow to ensure legibility over any tile colour. The title reads “London Underground” in body-text colour and “What-If Simulator” in the current accent colour.

Mobile variant: wrapped in a frosted-glass pill (`borderRadius 18`, `backdrop blur 10px`) for legibility on small screens where the map is denser.

Position shift in What-If Mode: the top and left offsets increase from 16px to 85px each, making room for the corner-bracket decoration of the What-If overlay.

8.2 MapSidebar

A slide-in panel anchored to the left edge of the viewport. Width is 280px (or 320px on large desktop). It slides off-screen to the left (`left: -sidebarWidth`) when collapsed and on-screen (`left: 0`) when open, animated with `transition: left 0.3s ease`.

Background: semi-transparent dark navy (`rgba(15,23,42,0.8)`) with `backdrop-filter blur(8px)`, bordered on the right by a 1px border. This gives the panel a glass-morphism appearance that allows the map to remain partially visible through it.

Content is delegated to `MapControlPanelContent`, which renders a What-If Mode toggle switch, a line-status section, a train-display section (in normal mode), and the line-status list.

Mobile: replaced by `MobileControlSheet` which is a bottom-sheet overlay with the same content.

8.3 SidebarToggleButton

A circular floating action button (FAB) that opens/closes the sidebar. It travels with the sidebar edge when open, positioned to remain visible at all times. Its colour matches the current accent (blue or amber).

8.4 What-If Mode Toggle

A custom iOS-style toggle switch (51 × 31px) with a white circular thumb. Off state: grey track (`rgba(120,120,128,0.32)`). On state: track fills with the amber accent colour. The thumb translates from `left: 2` to `left: 22` with `transition: left 0.3s ease`.

This single toggle is the primary mode gate. Enabling it triggers the accent-colour shift, corner brackets, the What-If label, hiding live-feed status blocks and activating line/station closure interactions.

8.5 MapControlPanelContent - Line Status List

A scrollable list of all 11 TfL lines. Each row is a compound button/card showing:

- **Coloured dot** - 14px circle, line colour, with a faint line-colour glow box-shadow.

- **Line label** - the line's display name.
- **Status badge (right-aligned)** - in normal mode shows live delay descriptions from TfL; in What-If Mode shows Open / Closed / Partly Closed.

Normal mode: rows are non-interactive (cursor: default) unless there is a delay, in which case hovering (desktop) or tapping (mobile) expands the row to reveal the delay reason and additional info.

What-If Mode: every row becomes a clickable toggle. Clicking a line closes/reopens it. Partly-closed lines dim to opacity 0.9 with an amber-tinted background. A “Reset all closures” button (red, full-width, with a glow box-shadow) appears at the top of the panel only in What-If Mode.

8.6 MapSearchBox

Desktop: positioned top-right. Collapses to a pill button showing the search icon and “Search stations” text. Clicking it expands to a 320px-wide panel with a text input, results list, and (when a station is focused) a station-action card.

Mobile: always visible as a full-width bar at the top (below the title pill), at y = 84px. Never collapses. Results appear as a scrollable list in a frosted overlay below the bar.

Station action card (focused station): shows two info chips (Start / Destination displaying the current route start/end), “Set as start” and “Set as destination” buttons, and a “Close station” / “Reopen station” button in What-If Mode.

Keyboard navigation: ArrowUp / ArrowDown move the selection index, Enter selects the highlighted station, Escape dismisses the panel on desktop. (Enter-selects-first-match behaviour is verified by MapSearchBox.test.js.)

8.7 MapHud

A heads-up display in the bottom-left showing live map zoom level, latitude/longitude, and a Reset View button. Uses Geist Mono for the coordinate readout. Positioned bottom-left and shifts right when the sidebar is open. The Reset View button also clears closures, route and search, providing a single escape to a clean state.

8.8 RouteInfoPanel

Positioned bottom-right on desktop (right: 70px, bottom: 25px). On mobile it floats bottom-center (left: 16, right: 16). This panel has two states:

Collapsed state: a single-line pill showing route start → end, along with action buttons (“Clear Route”, “Undo Clear”, “View Route”). On mobile, also shows travel time, changes, and stop count inline.

Expanded state: a scrollable card (maxHeight 340px desktop / 52vh mobile) with a header row showing the route summary and a Close button; on mobile, three stat chips (Duration, Changes, Stops) plus a Reset map view button; and a line-by-line itinerary (groupedLegs) where each leg card shows the line colour dot, line name, from → to station names, stop count, and travel time.

If no route is selected, the panel shows contextual instructions. If a route was just cleared, it shows a “Last cleared” summary with an “Undo clear route” button.

8.9 MapWhatIfOverlay

Active only when What-If Mode is on (desktop only). Renders four corner-bracket decorations - each an 80 × 80px div with two thick (15px) amber borders on the appropriate two sides - and a large monospace “WHAT-IF” label in the top-right with a pulsing amber dot. The corner brackets transition

smoothly when the sidebar opens/closes via left/bottom position transitions. The dot pulses via a flash CSS keyframe animation.

8.10 MobileControlSheet

On mobile portrait, the desktop sidebar is hidden entirely and replaced by a bottom-sheet drawer. It slides up from the bottom of the screen when the `SidebarToggleButton` is tapped. Contains the same `MapControlPanelContent` as the desktop sidebar, plus an interaction-mode toggle (Plan Routes / Edit Closures) shown only in What-If Mode.

8.11 MobileInteractionModeBar

A persistent bottom bar visible only when What-If Mode is active on mobile. Displays the current interaction mode (Plan Routes or Edit Closures) as a pill selector so the user can switch without opening the full control sheet.

8.12 RoutingErrorBox

A dismissible error toast that appears over the route-panel position when a routing request fails (eg. no path exists because stations are closed). Styled with the danger colour scheme. Has a close (×) button, so the app explains an impossible journey rather than failing silently.

9. Map Layer Rendering

The map is a Leaflet.js instance with a custom dark tile layer (`MapLeafletChrome` injects CSS overrides to keep tile backgrounds transparent). All London-specific data is rendered as React-Leaflet layers on top of the base map. The overall visual effect is a dark, near-black base map with neon-bright coloured lines, white station dots, and small glowing train markers. Station marker radius scales with zoom (zoom 12 → 7px, up to zoom 16 → 15px), keeping the map readable at every zoom level.

Layer Component	What It Renders
EdgeLayer	Polylines for all TfL line segments. Each line uses its official colour with an 8px white outline + 4px coloured stroke for crisp visibility. Lines offset slightly at junctions to avoid overlap. Closed lines are amber and black.
StationLayer	Circle markers for all stations. Radius scales with zoom (7px at z12 → 15px at z16). Clicking a station focuses it in the search box (and toggles closure in What-If Mode). Live-closed stations get a red glow; hypothetically closed ones get a distinct style.
RouteLayer	When a route is calculated, the path segments are highlighted with a thicker, more opaque polyline in the line's colour, distinguishing the route from all other lines.
CanvasTrainLayer / TrainLayer	Live TfL train positions rendered as small circles in train-specific colours (slightly darker than line colours). <code>CanvasTrainLayer</code> uses an HTML Canvas element for performance with many trains. Individual <code>TrainLayer</code> uses SVG/Leaflet markers for interactive selection.

10. Interaction Design and UX Patterns

10.1 Two-mode architecture

The app operates in exactly two modes gated by a single toggle. This is an intentional UX choice to prevent ambiguity: actions on the map (station clicks, line toggles) have entirely different consequences in each mode and the design makes the current mode unmistakable through the accent-colour system and the What-If overlay.

10.2 Progressive disclosure

Panels start collapsed. The search box collapses to a pill. The route panel collapses to a summary bar. The sidebar is always off-screen until toggled. This keeps the map maximally visible while making controls readily accessible.

10.3 Contextual actions

The search-box station-action card changes its button labels and colours dynamically based on state: if the station is already the route start, the “Set as start” button appears highlighted and reads “Selected as start”. Closed stations show muted/disabled routing buttons with explanatory text. This reduces the need for separate modals or confirmation dialogs.

10.4 Undo pattern

When a route is cleared, the panel keeps a record of the last cleared route and offers a one-step “Undo Clear” action. This follows the common pattern of forgiving interactions without requiring a full undo stack. Closures themselves are reversible toggles, providing an easy escape from any scenario.

10.5 Status indicators

Every live data source (line status, train feed) has a coloured dot + text pill in the sidebar. Green = live data, amber = fallback/schedule data. The dot has a matching box-shadow glow for extra emphasis. This pattern communicates data freshness without any additional modals or alerts.

10.6 Delay expansion

In normal mode, line rows with active delays can be expanded to read the disruption reason and additional info. On desktop this is triggered by hover (750ms debounce). On mobile it is triggered by tap-to-toggle. This respects platform conventions - hover is not available on touch.

10.7 Animation and motion

All animations are purposeful and brief:

Animation	Implementation
Sidebar slide-in/out	transition: left 0.3s ease on the sidebar div.
What-If toggle thumb	transition: left 0.3s ease on the toggle thumb.
Search box collapse/expand	CSS opacity + transform translateY(-12px) with 180ms ease (mobile).

Animation	Implementation
Route panel slide-off (mobile)	transform: translateY(calc(100% + 24px)) with transition 220ms ease - slides the panel below viewport when sidebar opens.
What-If corner brackets	animation: fadeIn 0.3s ease on mount.
WHAT-IF pulsing dot	animation: flash 1s ease-in-out infinite.
Delay row expansion	animation: expandDown 0.3s ease (CSS keyframe).
Leaflet tile load	transition: opacity 160ms linear on .leaflet-tile.

11. Responsive Design

The app ships two distinct layouts: desktop/landscape and mobile portrait. There is no tablet-specific breakpoint; tablets in landscape are treated as desktop. The useViewport hook drives a desktop layout (map plus a fixed right-hand sidebar - 280px, or 320px on large desktops, with overlays offset to accommodate the open sidebar) and a mobile-portrait layout (sidebar replaced by a bottom control sheet and a mobile interaction-mode bar). Sidebar width on mobile is $\min(360, \text{viewport.width})$ to ensure it fills small screens without overflow.

Element	Desktop / Landscape	Mobile Portrait
Sidebar / Controls	Fixed left panel, 280–320px wide	Bottom sheet, full-width
Search box	Top-right, collapsible, 320px wide	Top center, always visible, full-width
Title	Plain text, top-left	Frosted pill, top-left beside hamburger
Route Panel	Bottom-right, 340px wide	Bottom, full-width minus 16px margins
What-If Overlay	4 corner brackets + WHAT-IF label	Corner brackets hidden; mode shift visible via accent colour only
Mode Bar	N/A	Bottom bar for Plan Routes / Edit Closures in What-If Mode
Route summary stats	Text only in route-panel header	3-chip grid (Duration, Changes, Stops) visible in panel

12. Accessibility Considerations

Targeting WCAG 2.1 AA as the design standard, the current implementation has foundational accessibility measures in place, with clear opportunities to improve.

12.1 Current provisions

- **Semantic HTML:** the root page uses a `<main>` element; all buttons use `<button>` elements; inputs have `aria-label` attributes (e.g. `aria-label="Search stations"`).
- **Keyboard navigation:** the search box supports full keyboard navigation (ArrowUp/Down, Enter, Escape); Radix primitives provide keyboard navigation for the control panels.

- **Focus management:** buttons have hover/active cursor styles; the toggle switch uses explicit outline: none with focus managed through the parent container.
- **Text shadow:** the title overlay uses a six-layer text-shadow to ensure readability over any map-tile colour.
- **Not colour-alone:** closed elements use both colour and a distinct marker/outline style, and the route panel states changes in text; the Live↔What-If shift is reinforced by labelling.

12.2 Known gaps

- **Colour contrast:** several text-colour combinations (e.g. #64748B on #0F172A) may not meet WCAG AA 4.5:1 for normal text. The amber line colour (#FFD300) on dark backgrounds should be verified.
- **Screen-reader support:** the Leaflet map canvas and station markers do not have ARIA labels for the interactive SVG/canvas elements; station click events would be inaccessible to non-mouse users.
- **Focus ring:** the CSS rule outline: none on buttons removes the native focus ring. A custom :focus-visible style should be added.
- **Motion:** the pulsing What-If dot and tile-fade animations do not respect prefers-reduced-motion.
- **Alt text:** the app icon set (PNG, SVG) has no programmatic alt attributes in the layout.

13. Design Rationale and Heuristics

The interface reflects several of Nielsen's usability heuristics:

- **Visibility of system status** - the HUD shows live zoom/centre; the feed-status object reports whether train data is live or fallback; line status shows current delays/closures.
- **Match with the real world** - official TfL line colours and a familiar map metaphor.
- **User control and freedom** - Reset view / Reset closures provide an easy escape from any scenario.
- **Error prevention and recovery** - closures are reversible toggles; the routing-error box explains when a journey is impossible rather than failing silently.
- **Recognition over recall** - controls are visible on the map surface; the mode is signalled by colour throughout.
- **Flexibility** - search-driven and map-tap-driven interaction both work; the mode bar tailors map taps to the current task on mobile.

14. Figma Moodboard and Design Evolution

The original Figma file established the directional brief. The implemented design closely follows the dark, data-dense aesthetic but has evolved in several areas:

Design Dimension	Moodboard Direction	Implemented State
Overall theme	Dark, immersive, mission-control	Fully implemented - deep navy/black bg, frosted-glass panels.
Accent colour	Blue (TfL-inspired)	Blue + dynamic amber shift for What-If Mode (not in original brief).

Design Dimension	Moodboard Direction	Implemented State
Map interaction	Click to close stations	Full closure system: stations, lines, partial closures, live status.
Typography	Clean, technical sans-serif	Geist typeface chosen - modern, screen-optimised.
Line colours	TfL official palette	Exact TfL hex values used throughout.
Mode distinction	Not specified	Amber accent shift + WHAT-IF overlay added during development.
Mobile layout	Not specified in moodboard	Fully separate mobile layout with bottom sheet, search bar, mode bar.

Original moodboard: <https://www.figma.com/file/nuw9BCsmE4yIkZqhpWC6zn>

Appendix A - Key File Index

File Path	Purpose
app/global.css	Design tokens (CSS variables, OKLCH palette), layout resets, Leaflet overrides.
tailwind.config.js	Tailwind colour/font aliases, darkMode: 'class', animate plugin.
app/components/map/MapCanvas.jsx	Root layout component; composes all overlays and map layers.
app/components/mapComponents/constants.js	TfL line colours, train colours, line labels, zoom levels.
app/components/layout/MapControlPanelContent.jsx	Shared control-panel content (sidebar + mobile sheet).
app/components/layout/MapSearchBox.jsx	Station search panel (desktop + mobile variants).
app/components/layout/RouteInfoPanel.jsx	Route results panel (collapsed + expanded states).
app/components/layout/MapWhatIfOverlay.jsx	Corner brackets and WHAT-IF label overlay.
app/components/mapLayers/EdgeLayer.jsx	TfL line polyline rendering.
app/components/mapLayers/StationLayer.jsx	Station circle-marker rendering.